FICHE APPRENANT · PROJET SUIVI D'INCIDENTS BACKBONE

Semaine 1

Cadrage métier et environnement

Durée : 5 jours Métier NOC · Outils du développeur · Git · Modélisation

Objectifs de la semaine

À la fin de la semaine, tu sais :

- expliquer ce qu'est un incident sur un réseau backbone et comment un NOC le traite ;
- installer et faire tourner un projet Laravel connecté à MySQL ;
- utiliser Git pour sauvegarder ton travail ;
- dessiner un modèle de données avant d'écrire la moindre ligne de code.

1. Le métier : comprendre avant de coder

Un **backbone** est le réseau cœur d'un opérateur. Il relie les grandes villes par des **liaisons** à fort débit (fibre optique, faisceaux hertziens, satellite). Ces liaisons arrivent dans des **sites** (POP, data centers, stations FH) qui hébergent des **équipements** : routeurs, switchs, multiplexeurs DWDM, firewalls.

Le **NOC** (*Network Operations Center*) surveille ce réseau 24h/24. Quand une alarme tombe, un technicien :

1. **déclare** un incident (que se passe-t-il, où, depuis quand ?) ;
2. le **qualifie** (gravité, catégorie) et l'**assigne** à quelqu'un ;
3. **trace** chaque action (appel au terrain, redémarrage, remplacement de carte…) ;
4. le **résout** puis documente la **cause racine** et la **solution**.

Terme	Signification
P1 – Critique	Coupure de service importante (ex. une ville entière isolée)
P2 – Majeure	Service dégradé, ou redondance perdue
P3 – Mineure	Pas d'impact client direct
Cycle de vie	Ouvert → En cours → Résolu → Clos
MTTR	*Mean Time To Restore* : durée moyenne entre le début d'un incident et son rétablissement

> **À retenir** — Un développeur qui ne comprend pas le métier écrit une application que personne n'utilise. Pose des questions, toujours.

2. Les outils du développeur

Outil	Rôle	Vérifier l'installation
PHP 8.3+	Le langage de Laravel	`php -v`
Composer	Gestionnaire de dépendances PHP	`composer -V`
MySQL 8	Base de données relationnelle	`mysql --version`
Git	Historique et sauvegarde du code	`git --version`
VS Code	Éditeur (extensions : PHP Intelephense, Laravel Blade)	—

Comment fonctionne une requête dans Laravel

```
Navigateur  →  routes/web.php  →  Contrôleur  →  Modèle (MySQL)
                                        ↓
Navigateur  ←  HTML  ←  Vue Blade  ←────┘
```

Quand tu tapes `http://localhost:8000/incidents`, Laravel cherche la **route** `/incidents`, appelle la méthode du **contrôleur** associée, qui récupère les données via un **modèle** et les passe à une **vue** Blade qui produit le HTML.

3. MySQL : premiers pas en ligne de commande

Se connecter :

```
mysql -u root -p
```

Bonne pratique : **ne pas** utiliser `root` dans une application. On crée une base et un utilisateur dédiés :

```
CREATE DATABASE suivi_incident CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
CREATE USER 'suivi'@'localhost' IDENTIFIED BY 'un_mot_de_passe_solide';
GRANT ALL PRIVILEGES ON suivi_incident.* TO 'suivi'@'localhost';
FLUSH PRIVILEGES;
```

Quelques commandes utiles :

```
SHOW DATABASES;
USE suivi_incident;
SHOW TABLES;
DESCRIBE users;
```

> **Pourquoi `utf8mb4` ?** C'est le vrai UTF-8 complet de MySQL : accents, caractères spéciaux et emojis sont stockés sans erreur. `utf8` tout court est une version incomplète à éviter.

4. Git : sauvegarder son travail

```
git init
git add .
git commit -m "Initialise le projet Laravel"
git log --oneline
```

Règle d'or : **un commit = une petite étape qui fonctionne**, avec un message qui dit ce que fait le commit (« Ajoute la page de connexion », pas « modifs »).

Le fichier `.env` contient des mots de passe : il est déjà dans `.gitignore` et ne doit **jamais** être commité.

5. Modéliser avant de coder

Un **MCD** (modèle conceptuel de données) décrit les entités, leurs attributs et leurs relations. Exemple : *un site possède plusieurs équipements, un équipement appartient à un seul site* → relation **1-N**, la clé étrangère `site_id` est dans `equipements`.

Travail demandé

1. Installer PHP, Composer, MySQL, Git et VS Code.
2. Créer le projet : `composer create-project laravel/laravel suivi-incident`.
3. Créer la base et l'utilisateur MySQL dédiés (section 3).
4. Configurer `.env` : `DB_CONNECTION=mysql`, `DB_DATABASE`, `DB_USERNAME`, `DB_PASSWORD`, et `APP_LOCALE=fr`.
5. Lancer `php artisan migrate` puis `php artisan serve` ; ouvrir `http://localhost:8000`.
6. Dans MySQL, exécuter `SHOW TABLES;` et `DESCRIBE users;` : que vois-tu ? Qui a créé ces tables ?
7. Initialiser Git et faire le premier commit.
8. **Dessiner le modèle de données** (papier ou draw.io) avec au minimum :
   - `users` (nom, e-mail, rôle) ;
   - `sites` (code, nom, ville, type) ;
   - `equipements` (hostname, type, constructeur, IP de management, statut) — appartient à un site ;
   - `liaisons` (code, type, capacité, statut) — relie un **site A** et un **site B** ;
   - `incidents` (titre, description, sévérité, statut, catégorie, début, rétablissement, cause racine, solution) — rattaché à un site, éventuellement à un équipement **ou** une liaison, déclaré par un utilisateur, assigné à un utilisateur ;
   - `suivis` (journal des actions sur un incident).
9. Lister en 10 lignes maximum les écrans de l'application.

Livrable

- Le projet démarre sur `http://localhost:8000` et est connecté à MySQL.
- Le schéma du modèle de données (photo ou PNG dans `docs/`).
- La liste des écrans.

Auto-évaluation

- [ ] `php artisan migrate` s'exécute sans erreur sur MySQL
- [ ] J'utilise un utilisateur MySQL dédié, pas `root`
- [ ] Mon dépôt Git contient au moins un commit, sans le fichier `.env`
- [ ] Mon schéma contient les 6 tables et leurs cardinalités
- [ ] La liaison « entre deux sites » a bien deux clés étrangères vers `sites`
- [ ] Je sais expliquer P1/P2/P3, le cycle de vie et le MTTR à l'oral

Ressources

- Documentation Laravel — Installation : laravel.com/docs/installation
- Documentation Laravel — Structure des dossiers : laravel.com/docs/structure
- MySQL — CREATE USER et GRANT : dev.mysql.com/doc/refman/8.4/en/create-user.html
- Git en 15 minutes : learngitbranching.js.org

Pour aller plus loin

Chercher la différence entre un **incident** et un **problème** au sens ITIL, et noter en 3 lignes ce que ça changerait dans l'application.